

Python Core & OOPs Interview Questions and Answers

A comprehensive, production-grade guide covering **Python Fundamentals, Object-Oriented Programming (OOPs), Exception Handling, File Handling, and Serialization**. Designed for quick reference, technical interviews, and GitHub repository hosting.

Table of Contents

1. [Classes and Objects](#1-classes-and-objects)
2. [Four Pillars & Core Concepts of OOPs](#2-four-pillars--core-concepts-of-oops)
3. [Constructors](#3-constructors)
4. [Encapsulation & Access Modifiers](#4-encapsulation--access-modifiers)
5. [Inheritance, MRO & The Diamond Problem](#5-inheritance-mro--the-diamond-problem)
6. [Polymorphism & Method Overriding](#6-polymorphism--method-overriding)
7. [The `super()` Keyword](#7-the-super-keyword)
8. [Abstraction & Abstract Base Classes (ABC)](#8-abstraction--abstract-base-classes-abc)
9. [Exception Handling](#9-exception-handling)
10. [File Handling](#10-file-handling)
11. [Serialization & Deserialization](#11-serialization--deserialization)
12. [Object Inspection & Built-in Utilities](#12-object-inspection--built-in-utilities)

1. Classes and Objects

Q1. What is a class?

Answer:

A **class** is a user-defined blueprint or template used to create objects. It encapsulates data (attributes/properties) and functions (methods) that operate on that data into a single logical unit.

```
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model
```

Q2. What is an object?

Answer:

An **object** is an instance of a class. When a class is defined, no memory is allocated until an object of that class is instantiated. It holds actual values for the properties defined in the class.

```
my_car = Car("Tesla", "Model 3") # 'my_car' is an object
```

Q3. What is the difference between a class and an object?

Answer:

Feature	Class	Object
Definition	Blueprint/Template for objects Instance of a class	
Memory Allocation	Occupies no memory at creation (only code object)	Allocates memory on the heap when created
Existence	Logical entity	Physical entity
Example	<code>class Car:</code> <code>my_car = Car()</code>	

Q4. Why do we create objects?

Answer:

Objects allow us to:

1. **Manage State:** Store specific data unique to that entity.
2. **Execute Behavior:** Call class methods operating on that specific state.
3. **Reusability & Modular Code:** Create thousands of unique entities using a single class structure without code duplication.

Q5. Can we create multiple objects from a single class?

Answer:

Yes. A class acts as a reusable blueprint. You can instantiate as many distinct objects as memory permits, each maintaining its own state.

```
car1 = Car("Toyota", "Corolla")
car2 = Car("Ford", "Mustang")
```

Q6. Where are objects stored?

Answer:

In Python, objects are stored in **Heap Memory**. The variable holding the object stores a **reference (memory address)** pointing to that heap address. Python's memory manager handles dynamic allocation and garbage collection (using reference counting and a generational cyclic garbage collector).

Q7. What is the purpose of the `self` keyword?

Answer:

`self` represents the **instance of the class**. It allows methods to access and modify attributes and other methods belonging to that specific instance.

```
class Dog:
    def __init__(self, name):
        self.name = name # Binds name to the specific instance
```

Q8. Is `self` a keyword in Python?

Answer:

No. `self` is **not a reserved keyword** in Python; it is a strong positional convention. You can rename it to anything (e.g., `this`, `me`), but sticking to `self` is standard PEP 8 style guidelines.

Q9. Why is `self` passed as the first parameter to instance methods?

Answer:

When an instance method is called on an object (`obj.method(arg)`), Python automatically translates it under the hood to `Class.method(obj, arg)`. Therefore, the instance itself must be explicitly received as the first parameter by convention named `self`.

Q10. Can we rename `self` ? If yes, should we?

Answer:

- **Can we?** Yes. Python allows any valid identifier name as the first parameter.
 - **Should we?** **No.** Replacing `self` breaks PEP 8 conventions, reduces readability, and irritates team members and static analysis tools.
-

2. Four Pillars & Core Concepts of OOPs

Q11. What are the four pillars of Object-Oriented Programming (OOP)?

Answer:

1. **Encapsulation:** Bundling data and methods into a single unit and restricting direct external access.
 2. **Abstraction:** Hiding complex background implementation details and showing only relevant features.
 3. **Inheritance:** Enabling a child class to inherit attributes and methods from a parent class.
 4. **Polymorphism:** Allowing a single interface/method name to behave differently depending on the object or data type calling it.
-

Q12. What is polymorphism? Explain with an example.

Answer:

Polymorphism ("many forms") is the ability of different classes to respond to the same method call in their own specific way.

```

class Animal:
    def make_sound(self):
        pass

class Dog(Animal):
    def make_sound(self):
        return "Woof!"

class Cat(Animal):
    def make_sound(self):
        return "Meow!"

def play_sound(animal: Animal):
    print(animal.make_sound())

play_sound(Dog()) # Output: Woof!
play_sound(Cat()) # Output: Meow!

```

Q13. What is Duck Typing in Python?

Answer:

Duck Typing is Python's dynamic typing philosophy: **"If it walks like a duck and quacks like a duck, it's a duck."**

Python does not verify an object's type explicitly; instead, it checks if the required method or attribute exists at runtime.

```

class Duck:
    def fly(self):
        print("Duck flying")

class Airplane:
    def fly(self):
        print("Airplane flying")

def lift_off(entity):
    entity.fly() # Works for both Duck and Airplane without type restrictions

lift_off(Duck())
lift_off(Airplane())

```

Q14. What is Dynamic Binding?

Answer:

Dynamic Binding (also known as Late Binding) means that the decision about which specific method implementation to invoke is made at **runtime**, rather than at compile time. In Python, all method calls are dynamically bound.

Q15. What is Operator Overloading? Give an example.

Answer:

Operator Overloading allows custom classes to define or redefine the behavior of built-in operators (`+`, `-`, `*`, `==`, etc.) by implementing magic/dunder methods (e.g., `__add__`, `__sub__`, `__eq__`).

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)

p1 = Point(1, 2)
p2 = Point(3, 4)
p3 = p1 + p2 # Calls p1.__add__(p2)
print(p3.x, p3.y) # Output: 4 6
```

3. Constructors

Q16. What is a constructor?

Answer:

A **constructor** is a special method inside a class automatically executed when a new object of that class is instantiated. In Python, the `__init__()` method acts as the primary initializer/constructor.

Q17. What are the types of constructors in Python?

Answer:

1. **Default Constructor:** Created automatically by Python if no constructor is provided, taking no parameters besides `self`.
2. **Parameterized Constructor:** Accepts custom arguments supplied during instantiation to initialize object attributes.

Q18. Can a class have multiple constructors?

Answer:

No, not natively through method overloading. Python keeps only the *last defined* `__init__` method in a class. However, multiple initialization approaches can be achieved using default parameters, `*args` / `kwargs`, or Class Methods (`@classmethod`)** as alternative constructors.

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    @classmethod
    def from_birth_year(cls, name, birth_year):
        return cls(name, 2026 - birth_year) # Alternative constructor
```

Q19. What happens if a constructor is not defined?

Answer:

Python uses an implicit default constructor inherited from the base `object` class. It initializes the object without creating any custom instance variables.

Q20. What is the difference between a constructor and a normal method?

Answer:

Feature	Constructor (<code>__init__</code>)	Normal Method
Execution	Called automatically upon instantiation Called explicitly using object reference	
Purpose	Initializes instance attributes Implements business logic or actions	
Return Value	Must always return <code>None</code>	Can return any valid Python object/value

Q21. Can a constructor return a value?

Answer:

No. A constructor must return `None`. Returning any non-`None` value from `__init__()` raises a `TypeError: __init__() should return None`.

Q22. Can we call a constructor explicitly?

Answer:

Yes. You can call `obj.__init__()` or `ClassName.__init__(obj)` explicitly, though it re-initializes the state of the object rather than creating a new instance.

Q23. Can a constructor call another method?

Answer:

Yes. Inside `__init__`, you can call instance methods or class methods using `self.method_name()`.

```
class Account:
    def __init__(self, initial_balance):
        self.balance = initial_balance
        self.validate()

    def validate(self):
        if self.balance < 0:
            raise ValueError("Balance cannot be negative")
```

Q24. What is constructor chaining?

Answer:

Constructor Chaining is the process of calling a parent class constructor from a child class constructor, ensuring base attributes are properly initialized before child attributes are added.

```
class Parent:
    def __init__(self, name):
        self.name = name

class Child(Parent):
    def __init__(self, name, age):
        super().__init__(name) # Constructor Chaining
        self.age = age
```

Q25. What happens if the parent class has a parameterized constructor?

Answer:

If the parent class has a parameterized constructor, the child class constructor must explicitly invoke `super().__init__(args...)` passing the required arguments. If omitted, instantiation of the child class will fail if default arguments aren't present.

4. Encapsulation & Access Modifiers

Q26. What is encapsulation?

Answer:

Encapsulation is the wrapping of data (attributes) and code (methods) into a single unit while restricting direct access to internal details from outside the class.

Q27. Why do we need encapsulation?

Answer:

1. **Data Protection:** Prevents unintended direct modification of internal state.
2. **Validation:** Ensures values passed through setters meet business rules.
3. **Maintainability:** Internal implementation can change without breaking public APIs.

Q28. Explain public, protected, and private members.

Answer:

Python enforces access levels using naming conventions:

1. **Public (`name`):** Accessible from anywhere (inside or outside the class).
2. **Protected (`_name`):** Intended to be accessible only within the class and its subclasses (convention-based, not enforced strictly).
3. **Private (`__name`):** Intended to be accessible *only* within the defining class. Python enforces this via **Name Mangling**.

Q29. Does Python truly support private variables?

Answer:

No. Python does not enforce hard access control like Java or C++. Private members prefixed with `__` undergo **Name Mangling** to prevent accidental overrides, but they remain accessible if you know the mangled name.

Q30. What is Name Mangling?

Answer:

Name Mangling is Python's mechanism for transforming private attribute names. Any variable named `__attribute` inside class `MyClass` is internally rewritten to `__MyClass__attribute`.

```
class Bank:
    def __init__(self):
        self.__balance = 1000

b = Bank()
# print(b.__balance) # Raises AttributeError
print(b._Bank__balance) # Output: 1000 (Mangled access)
```

Q31. How can we access private variables?

Answer:

1. Via Public Getters/Setters or `@property` decorators (Recommended):

```
class User:
    def __init__(self, password):
        self.__password = password

    @property
    def password(self):
        return "*****" # Masked access
```

2. Via Name Mangling (Discouraged): `obj._ClassName__variable`

Q32. What is the difference between encapsulation and abstraction?

Answer:

Concept	Focus	Implementation
Encapsulation		

How to hide internal data/state | Private/Protected members, Getters/Setters |

| **Abstraction** | **What** functionality is offered (hiding implementation complexity) | Abstract Classes (`ABC`), Interfaces |

Q33. What are the advantages of encapsulation?

Answer:

- Prevents accidental data corruption.
- Improves security by forcing interaction through defined interfaces.
- Allows internal refactoring without affecting external code dependent on the class.

5. Inheritance, MRO & The Diamond Problem

Q34. What is inheritance?

Answer:

Inheritance is a mechanism where a child class (subclass) inherits attributes and methods from a parent class (superclass), promoting code reuse and establishing hierarchical relationships.

Q35. Why do we use inheritance?

Answer:

- **Code Reusability:** Avoid duplicating common logic across multiple classes.
 - **Extensibility:** Extend functionality of existing classes without modifying them.
 - **Polymorphic Behavior:** Treat sub-objects uniformly through base class interfaces.
-

Q36. What are the different types of inheritance?

Answer:

1. **Single Inheritance:** Child inherits from one Parent.
 2. **Multiple Inheritance:** Child inherits from multiple Parents.
 3. **Multilevel Inheritance:** Class C inherits from Class B, which inherits from Class A.
 4. **Hierarchical Inheritance:** Multiple Child classes inherit from one Parent.
 5. **Hybrid Inheritance:** Combination of two or more types of inheritance.
-

Q37. Does Python support multiple inheritance?

Answer:

Yes. In Python, a single class can inherit directly from more than one parent class.

```
class A: pass
class B: pass
class C(A, B): pass # Multiple Inheritance
```

Q38. What is multilevel inheritance?

Answer:

A linear chain of inheritance where a derived class acts as a base class for another class.

```
class Grandparent: pass
class Parent(Grandparent): pass
class Child(Parent): pass
```

Q39. What is hierarchical inheritance?

Answer:

When multiple derived classes inherit from the same base class.

```
class Parent: pass
class Child1(Parent): pass
class Child2(Parent): pass
```

Q40. What is hybrid inheritance?

Answer:

A mix of two or more forms of inheritance (e.g., combining Multiple and Hierarchical inheritance).

Q41. What is the difference between inheritance and composition?

Answer:

- **Inheritance ("IS-A" relationship):** Subclass is a specialized version of the base class (`Car` is a `Vehicle`).
- **Composition ("HAS-A" relationship):** Class contains an instance of another class (`Car` has an `Engine`).

```
# Composition Example (Preferred for loose coupling)
class Engine:
    def start(self): print("Engine started")

class Car:
    def __init__(self):
        self.engine = Engine() # HAS-A relationship
```

Q42. What is a real-life example of inheritance?

Answer:

A base class `User` containing `email` and `login()` inherited by specialized classes `AdminUser` and `CustomerUser` .

Q43. When should inheritance NOT be used?

Answer:

- When there is no clear "IS-A" relationship.
- When you only need code reuse without sharing an identity (use **Composition** instead).
- When inheritance hierarchies become overly deep, making code brittle and hard to maintain.

Q44. Why can multiple inheritance become complicated?

Answer:

It can lead to ambiguity issues like method naming collisions, state duplications, and the **Diamond Problem** (determining which parent method to invoke when multiple parents override the same method).

Q45. What is Method Resolution Order (MRO)?

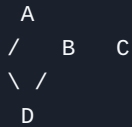
Answer:

MRO is the deterministic algorithm Python uses to decide the order in which base classes are searched when looking for a method or attribute. You can view it using `ClassName.mro()` or `ClassName.__mro__` .

Q46. What is the Diamond Problem?

Answer:

The **Diamond Problem** occurs in multiple inheritance when a class **D** inherits from **B** and **C**, both of which inherit from a common base class **A**. If **B** and **C** override a method from **A**, which version should **D** execute?



Q47. How does Python solve the Diamond Problem?

Answer:

Python solves the Diamond Problem using **Method Resolution Order (MRO)** powered by the **C3 Linearization Algorithm**, ensuring methods are invoked in a strict, left-to-right, depth-first manner without duplicating execution of base classes.

Q48. Explain C3 Linearization.

Answer:

C3 Linearization guarantees three essential properties:

1. **Preserving Local Precedence Order:** Child classes are searched before parents, and parents in left-to-right order as specified in definition.
2. **Monotonicity:** Subclasses preserve the inheritance order of their parent classes.
3. **No Duplicate Execution:** Ensures no base class is evaluated more than once.

6. Polymorphism & Method Overriding

Q49. What is method overriding?

Answer:

Method Overriding occurs when a child class provides a specific implementation for a method that is already defined in its parent class.

```
class Parent:
    def show(self):
        print("Parent implementation")

class Child(Parent):
    def show(self):
        print("Child implementation") # Overrides parent method
```

Q50. Why is method overriding used?

Answer:

To allow child classes to provide specialized behavior while maintaining a uniform method signature defined by the parent class.

Q51. What is the difference between method overriding and method overloading?

Answer:

Feature	Method Overriding	Method Overloading
Definition	Redefining a parent class method in a child class Defining multiple methods with same name but different signatures in same class	
Occurrence	Across inherited classes Within the same class	
Python Support	Fully supported natively Not supported natively (last definition overrides prior ones)	

Q52. Does Python support method overloading?

Answer:

Not natively in the traditional sense. Defining multiple methods with the same name overwrites earlier definitions. However, it can be simulated using:

1. Default arguments or `*args` / `**kwargs`.
2. The `@functools.singledispatch` decorator.

```
# Simulating Overloading
class Calculator:
    def add(self, a, b, c=0):
        return a + b + c
```

Q53. What happens if a child class doesn't override a method?

Answer:

The child class automatically inherits and executes the implementation provided in the parent class.

Q54. Can constructors be overridden?

Answer:

Yes. If a child class defines its own `__init__`, it overrides the parent's `__init__`. To maintain parent initialization, call `super().__init__()`.

Q55. Can private methods be overridden?

Answer:

No. Private methods (prefixed with `__`) are name-mangled inside their defining class. A child class creating a method with the same name creates a separate mangled method, not an override.

Q56. What is runtime polymorphism?

Answer:

Runtime Polymorphism means that the target method to execute is resolved dynamically at runtime based on the actual object instance calling it, rather than at compile time. Method overriding is a classic example.

7. The `super()` Keyword

Q57. What is `super()` ?

Answer:

`super()` is a built-in proxy object that delegates method calls to a parent or sibling class in the class's **Method Resolution Order (MRO)**.

Q58. Why do we use `super()` ?

Answer:

- Avoids hardcoding parent class names.
 - Essential for cooperative multiple inheritance.
 - Ensures parent initialization code is cleanly executed.
-

Q59. Can `super()` be used to access parent variables?

Answer:

- **Methods & Properties:** Yes (`super().method_name()`).
 - **Instance Variables:** No direct access. Instance variables created in `__init__` live on `self`. Calling `super().__init__()` binds them directly to `self`.
-

Q60. Can we use the parent class name instead of `super()` ?

Answer:

Yes, but it is bad practice:

`ParentClass.method(self)` hardcodes the relationship and breaks dynamic cooperative multiple inheritance in complex MRO chains.

Q61. How does `super()` work in multiple inheritance?

Answer:

It calls the **next class in the MRO chain**, not necessarily the immediate direct parent written in the class declaration. This enables **cooperative method calls**.

Q62. What are the advantages of using `super()` ?

Answer:

- Maintains clean, maintainable inheritance trees.
 - Prevents redundant execution of shared base class constructors in complex multiple inheritance hierarchies.
-

8. Abstraction & Abstract Base Classes (ABC)

Q63. What is abstraction?

Answer:

Abstraction is the concept of hiding background implementation details and exposing only essential interface features to the user.

Q64. What is the difference between abstraction and encapsulation?

Answer:

- **Abstraction:** Solves design-level problems ("What does this object do?").
 - **Encapsulation:** Solves implementation-level problems ("How do I protect internal state?").
-

Q65. What is an abstract class?

Answer:

An **abstract class** is a class that cannot be instantiated directly and serves as a blueprint containing one or more abstract methods that subclasses must implement.

Q66. Can we create an object of an abstract class?

Answer:

No. Attempting to instantiate an abstract class that contains abstract methods raises a `TypeError`.

Q67. What is an abstract method?

Answer:

An **abstract method** is a method declared with the `@abstractmethod` decorator without an implementation (or only a placeholder) in the base class. Derived classes **must** override it.

Q68. Why do we use abstract classes?

Answer:

To enforce a strict contract across all subclasses, ensuring that derived classes implement required methods and standardizing application structure.

Q69. Which module provides abstract class support?

Answer:

The `abc` module (Abstract Base Classes).

```
from abc import ABC, abstractmethod

class PaymentGateway(ABC):
    @abstractmethod
    def process_payment(self, amount):
        pass
```

Q70. What is the difference between an interface and an abstract class?

Answer:

In Python:

- **Abstract Class:** Can contain both concrete methods (with implementation) and abstract methods, as well as instance variables.
 - **Interface:** Python does not have a separate `interface` keyword. Interfaces are created using pure abstract classes containing **only** abstract methods and no concrete logic.
-

Q71. Can an abstract class have constructors, variables, and concrete methods?

Answer:

Yes. An abstract class can have `__init__`, state attributes, and normal (concrete) methods alongside abstract methods.

Q72. When should we choose an abstract class?

Answer:

Choose an abstract class when multiple closely related classes share common logic/implementation, but also require unique class-specific behavior for mandatory interfaces.

9. Exception Handling

Q73. What is an exception?

Answer:

An **exception** is an event or error that occurs during program execution that breaks the normal flow of instructions.

Q74. What is the difference between a syntax error and an exception?

Answer:

- **Syntax Error:** Caused by invalid Python code structure; detected by the parser **before** program execution starts.
 - **Exception:** Occurs at **runtime** when syntactically correct code encounters an unexpected operation (e.g., `1 / 0`).
-

Q75. What is the purpose of the `try` block?

Answer:

The `try` block wraps code that might potentially raise an exception at runtime.

Q76. What is the purpose of the `except` block?

Answer:

The `except` block contains the recovery or error-handling code executed if a matching exception occurs inside the `try` block.

Q77. What is the purpose of the `else` block?

Answer:

The `else` block executes **only if NO exceptions were raised** inside the `try` block.

Q78. What is the purpose of the `finally` block?

Answer:

The `finally` block **always executes**, regardless of whether an exception occurred, was caught, or was uncaught. It is primarily used for cleanup tasks (closing files, DB connections).

Q79. When does the `finally` block execute?

Answer:

It executes in all scenarios—even if `return`, `break`, or `continue` statements are executed inside the `try` or `except` blocks.

Q80. What is the difference between `raise` and `assert` ?

Answer:

- **`raise`** : Explicitly triggers an exception at any point in code execution.
 - **`assert`** : Debugging tool that checks if a condition is `True` . If `False` , it raises an `AssertionError` . (Can be disabled globally with `-O` flag).
-

Q81. Can we write multiple `except` blocks?

Answer:

Yes. You can chain multiple `except` blocks to handle different types of exceptions specifically.

```
try:
    val = int(input())
    res = 10 / val
except ValueError:
    print("Invalid integer")
except ZeroDivisionError:
    print("Cannot divide by zero")
```

Q82. What is the difference between `Exception` and `BaseException` ?

Answer:

- **`BaseException`** : The root hierarchy class for all built-in exceptions. Includes system-exiting signals (`SystemExit` , `KeyboardInterrupt`).
 - **`Exception`** : Inherits from `BaseException` . It is the superclass for all application/user-level non-system-exiting exceptions.
-

Q83. What is the difference between an Error and an Exception?

Answer:

- **Errors (`SystemExit` , `MemoryError`)**: Critical system failures that applications usually should not attempt to catch.
 - **Exceptions (`ValueError` , `KeyError`)**: Conditions that an application can reasonably catch and recover from.
-

Q84. What is a custom exception?

Answer:

A user-defined exception class created by inheriting from the built-in `Exception` class.

```
class InsufficientFundsError(Exception):  
    pass
```

Q85. Why do we create custom exceptions?

Answer:

To provide domain-specific error handling and make debugging clearer across enterprise codebases.

Q86. What is the difference between `ValueError` and `TypeError` ?

Answer:

- `TypeError` : Operation applied to an inappropriate data type (e.g., `'5' + 5`).
- `ValueError` : Right data type, but an invalid/unacceptable value (e.g., `int('abc')`).

Q87. Why should we catch specific exceptions instead of using a generic `except:` ?

Answer:

Bare `except:` catches *everything*, including `KeyboardInterrupt` (`Ctrl+C`) and `SystemExit` , masking critical system behavior and hiding unexpected bugs.

10. File Handling

Q88. Why do we use files?

Answer:

Files allow persistent data storage beyond program execution lifetime (RAM data cleared on termination).

Q89. What are the different file opening modes?

Answer:

- `'r'` : Read (default). Opens file for reading.
- `'w'` : Write. Overwrites or creates a new file.
- `'a'` : Append. Adds data at end of file.
- `'x'` : Exclusive creation. Fails if file exists.
- `'b'` : Binary mode (e.g., `'rb'` , `'wb'`).
- `'+'` : Updating mode (Read + Write, e.g., `'r+'` , `'w+'`).

Q90. What is the difference between `r` and `r+` ?

Answer:

- `r` : Read-only mode.
- `r+` : Read and Write mode. File pointer is placed at the start without truncating content.

Q91. What is the difference between `w` and `a` ?

Answer:

- `w` : Truncates (erases) existing file contents before writing.
 - `a` : Appends new data to the end of the file, keeping existing content intact.
-

Q92. What is the difference between text mode and binary mode?

Answer:

- **Text Mode (`'t'`)**: Handles string objects, performing automatic line-ending translations (`\n` vs `\r\n`).
 - **Binary Mode (`'b'`)**: Handles raw bytes (`bytes` objects) without modifying line endings or encoding.
-

Q93. What does the `seek()` method do?

Answer:

`seek(offset, whence)` moves the file pointer to a specified byte location within the file.

Q94. What does the `tell()` method do?

Answer:

`tell()` returns the current byte position of the file pointer.

Q95. What is the difference between `read()`, `readline()`, and `readlines()` ?

Answer:

- `read(n)` : Reads entire file (or `n` bytes/chars) into a single string/bytes object.
 - `readline()` : Reads a single line from the current position.
 - `readlines()` : Reads all lines into a **list of strings**.
-

Q96. Why should we use the `with` statement?

Answer:

`with` acts as a **Context Manager**. It guarantees that resources (like file handles) are automatically closed when exiting the block, even if an exception is thrown.

```
with open("data.txt", "r") as f:
    data = f.read()
```

Q97. What happens if a file is not closed?

Answer:

Unclosed files cause resource leaks, potential data loss due to un-flushed write buffers, and OS file locking issues.

Q98. What is buffering in file handling?

Answer:

Buffering holds data temporarily in memory before performing physical I/O disk writes, dramatically improving performance by minimizing disk interactions.

Q99. What is a text file?**Answer:**

A text file stores data as a sequence of human-readable characters encoded in UTF-8, ASCII, etc.

Q100. What is a binary file?**Answer:**

A binary file stores raw byte data not meant for direct text reading (e.g., images `.png`, executable files, compiled `.pyc`).

Q101. What is the difference between a CSV file and a TXT file?**Answer:**

- **TXT:** Unstructured or arbitrarily structured plain text file.
 - **CSV:** Tabular data formatted with specific column delimiters (usually commas).
-

Q102. What are the advantages of file handling?**Answer:**

- Persistent storage.
 - Sharing data across independent programs.
 - Bulk processing of large datasets without overwhelming RAM memory.
-

11. Serialization & Deserialization

Q103. What is serialization?**Answer:**

Serialization (Pickling/Marshalling) is the process of converting an in-memory Python object structure into a byte stream or format suitable for disk storage or network transmission.

Q104. What is deserialization?**Answer:**

Deserialization (Unpickling) is the reverse process: converting a serialized byte stream or string back into a live in-memory Python object structure.

Q105. Why is serialization required?**Answer:**

To save application state, cache object instances, send structured objects over sockets/HTTP APIs, or store application settings persistently.

Q106. What are the advantages of serialization?**Answer:**

- Preserves object state across program restarts.

- Enables distributed computing (e.g., sharing tasks across Celery workers).

Q107. What is the difference between Pickle and JSON?

Answer:

Feature	Pickle	JSON
Format	Python-specific binary format Language-agnostic plain text format	
Security	Unsafe with untrusted data (code execution risk)	Safe (data parsing only)
Data Types Supported	Almost all Python objects (custom classes, functions)	Basic types (dicts, lists, strings, numbers, booleans)

Q108. Why is JSON preferred over Pickle in many applications?

Answer:

JSON is human-readable, safe against arbitrary code execution, and compatible across almost all programming languages (Java, JavaScript, C++, Go, etc.).

Q109. What are the limitations of Pickle?

Answer:

- **Security Vulnerability:** Unpickling malicious data can execute arbitrary system commands.
- **Python Only:** Not easily parsed outside Python environments.

Q110. Can Pickle serialize every Python object?

Answer:

No. Objects like active database connections, open file handles, running network sockets, and lambda functions generally cannot be pickled natively.

Q111. Where is serialization commonly used?

Answer:

- Redis caching.
- Rest APIs (JSON serialization).
- Machine Learning model persistence (`joblib` or `pickle` for model binaries).

Q112. Explain a real-world use case of serialization.

Answer:

Saving a trained machine learning model (`scikit-learn` model) to disk using `joblib` / `pickle` so a web server can load it later to serve real-time predictions without retraining.

12. Object Inspection & Built-in Utilities

Q113. What is the difference between `isinstance()` and `type()` ?

Answer:

- `isinstance(obj, Class)` : Checks if `obj` is an instance of `Class` **or any subclass derived from it** (supports inheritance).
- `type(obj) == Class` : Checks for an **exact match** of the type, ignoring subclass inheritance relationships.

Q114. When should you use `isinstance()` instead of `type()` ?

Answer:

Always use `isinstance()` for type checks in production code because it respects polymorphic inheritance hierarchies.

Q115. What is the purpose of `issubclass()` ?

Answer:

`issubclass(Child, Parent)` tests if a class is a direct or indirect derived class of another specified class.

```
issubclass(bool, int) # Output: True (bool inherits from int in Python)
```

Q116. What is the difference between `dir()` and `help()` ?

Answer:

- `dir(obj)` : Returns a list of valid attribute and method names associated with `obj`.
- `help(obj)` : Opens documentation/docstrings detailing usage of `obj` and its methods.

Q117. How do you inspect an object's attributes and methods?

Answer:

1. `dir(obj)` : List attributes/methods.
2. `vars(obj)` or `obj.__dict__` : View instance attribute key-value pairs.
3. `inspect` module: Advanced inspection (e.g., getting source code).